CMPE3209 Computer Organization — Proje Teslim Tutanağı

Kullanım talimatı

Bu tutanak projenin tek resmi teslim belgesidir. Her kilometre taşında (M1, M2 ve final; tümü pazartesi) ilgili adım doldurulur ve aynı dosya güncellenerek yeniden teslim edilir. Ayrıntılı görev tanımları proje yönergesindedir; bu tutanak yalnızca ne istendiğini ve nereye yazılacağını gösterir.

- Tüm sayılar birimiyle yazılır (byte, cycle, ns, GHz). Hex değerler `0x` önekiyle yazılır.
- **Varsayım** olan her değer "(varsayım)" etiketiyle işaretlenir.
- Tablolara sığmayan çıktılar (assembly listesi, objdump, CSV izi, ekran görüntüleri) Git deposuna konur. Tutanakta dosya adıyla atıf yapılır.
- Boş bırakılan zorunlu alan, o adımın puanını sıfırlar.

Öğrenci bilgileri

Alan	Değer
Ad Soyad	Necati Selçuk
Öğrenci numarası (SID)	5001240039
E-posta	selcuk.necati@ogr.iu.edu.tr
Seçilen görev (G1–G5)	G2 — Sınav notu dağılımı
Seçilen ISA ve sürümü (ör. RV32IM)	RISC-V RV32IM. Gerekçe: M uzantısı çarpmayı mul ile yapar (rv32i'de kütüphane çağrısına dönüşürdü); sabit 32 bit encoding; RARS ile simüle edilebilir.
Derleyici ve sürümü	RISC-V gcc 16.1.0 (Compiler Explorer), -march=rv32im -mabi=ilp32
Simülatör / araçlar	RARS 1.6, VS Code, Git
Git deposu adresi	github.com/necatiselcuk/cmpe3209-project

Adım 0 — Kişiye özel parametreler

Bu adım M1 ile birlikte teslim edilir. Hatalı bir parametre, ona dayanan tüm adımları geçersiz kılar; öğretim üyesi bu tabloyu M1'de kontrol eder.

Sembol	Tanım	Hesap (adımlarıyla)	Sonuç
d	SID'in son hanesi	5001240039'un son hanesi	9
D2	SID'in son iki hanesi	5001240039'un son iki hanesi	39
S	SID'in hanelerinin toplamı	5+0+0+1+2+4+0+0+3+9 (10 hane; 9 hane kuralı yalnızca C sabiti için)	24
N	24 + (D2 mod 25)	24 + (39 mod 25) = 24 + 14	38 eleman (152 byte)
Cache	d'ye göre: boyut / line / mapping / line sayısı / set sayısı	d = 9; line sayısı = 128 / 16 = 8; set sayısı = 8 / 4 = 2	128 byte / 16 byte / 4-way set-assoc. (LRU) / 8 line / 2 set
Write policy	d çift → write-back, d tek → write-through	d = 9 tek → write-through	write-through; write-allocate (varsayım)
k	(S mod 7) + 3	(24 mod 7) + 3 = 3 + 3	6
Interrupt türü	S çift → Timer, S tek → I/O	S = 24 çift → Timer	Timer
f	1.0 + 0.1 × d GHz (varsayım)	1.0 + 0.1 × 9	1.9 GHz (varsayım)
T1 / T2	T1 = 1 cycle; T2 = 20 + 5 × (d mod 4) cycle (varsayım)	T1 = 1; T2 = 20 + 5 × (9 mod 4) = 20 + 5 × 1	T1 = 1 cycle, T2 = 25 cycle (varsayım)
Nc	2 + (S mod 7) (varsayım)	2 + (24 mod 7) = 2 + 3	5 (varsayım)
Görev parametresi	Seçilen göreve göre (T, P, K, yön veya R)	P = 50 + (1240039 mod 10) = 50 + 9	P = 59

Adres alanları (ISA'nın adres genişliği s + w bit; 64-bit ISA'da düşük 32 bit kullanılır):

Alan	Bit sayısı	Hesap
Word (w)	4	line boyu 16 byte = 2^4 → w = log2(16) = 4
Line (r) veya set (d)	1 (set, d)	set sayısı 2 = 2^1 → d = log2(2) = 1
Tag (s − r veya s − d)	27	s = 32 − w = 28; tag = s − d = 28 − 1 = 27
Toplam	32	27 + 1 + 4 = 32

Adım 1 — Görev seçimi ve C programı (M1: 2 Kasım, Pazartesi)

1.1 Görev seçimi (yalnızca birini işaretleyin):

- ☐ G1 Biyomedikal sinyal alarmı
- X G2 Sınav notu dağılımı
- ☐ G3 Mesaj şifreleme ve bütünlük
- ☐ G4 Sıralama ve medyan
- ☐ G5 Run-length sıkıştırma

1.2 Çözüm tasarımı (3–5 cümle: hangi algoritma, hangi fonksiyonlar, hangi veri yerleşimi ve neden): Programı üç döngüyle yazdım. Birinci döngü, LCG ile SID'den 38 not üretip global grades dizisine yazar. İkinci döngü her notu bucket_of fonksiyonuyla beş aralıktan birine sayar, 59 ve üstü notları pass_count'a ekler ve notları toplar; döngüden sonra mean = sum / N ile tamsayı ortalama bulunur. Üçüncü döngü her notu mark fonksiyonuna verir; not ortalamanın altındaysa bit 8 set edilir (OR 0x100) ve below_count bir artar. pass_count'ı ikinci döngüde saydım, çünkü işaretlenen bir not 256'dan büyük olur ve yanlışlıkla geçmiş sayılırdı; bütün sonuçlar global değişkenlerde tutulur.

1.3 C kaynak kodu. Satır numaralı kod aşağıdaki bloka yapıştırılır (25–60 satır):

```
1  /* Necati Selcuk, 5001240039, G2 - Exam grade distribution */
2
3  #define N 38
4  const int SID = 1240039;
5
6  int grades[N];
7  int bucket[5];
8  int mean;
9  int below_count;
10  int pass_count;
11
```

```c
/* returns range index 0..4 */
int bucket_of(int v)
{
    if (v < 50) return 0;
    if (v < 65) return 1;
    if (v < 75) return 2;
    if (v < 85) return 3;
    return 4;
}

/* flags grade with bit 8 if below mean */
int mark(int v, int m)
{
    if (v < m) return v | 0x100;
    return v;
}

int main(void)
{
    unsigned int x = SID;
    int sum = 0;

    /* pass 1: generate grades */
    for (int i = 0; i < N; i++) {
        x = 1664525u * x + 1013904223u;
        grades[i] = (x >> 16) % 101;
    }

    /* pass 2: count ranges, passing grades and sum */
    for (int i = 0; i < N; i++) {
        int v = grades[i];
        bucket[bucket_of(v)]++;
        if (v >= 59) pass_count++;
        sum += v;
    }
    mean = sum / N;

    /* pass 3: flag grades below mean */
    for (int i = 0; i < N; i++) {
        int f = mark(grades[i], mean);
        if (f != grades[i]) below_count++;
        grades[i] = f;
    }

    return 0;
}
```

1.4 Zorunlu yapı eşlemesi. Her madde için satır numarası ve tek cümlelik açıklama:

#	Zorunlu yapı	Satır no	Açıklama
Y1	N elemanlı global int dizisi, en az iki geçiş	6; 35–38, 41–46, 50–54	grades[38] global dizisi üç döngüde baştan sona geçilir.
Y2	Koşullu dallanma	15–18, 25, 44, 52	bucket_of ve mark içindeki if'ler ile 44. ve 52. satırdaki if'ler.
Y3	Parametreli, değer döndüren fonksiyon çağrısı	13–20 ve 43; 23–27 ve 51	bucket_of(v) ve mark(v, m) parametre alıp değer döndürür.
Y4	Toplama/çıkarma, çarpma veya shift, bit işlemi	36, 37, 45, 25	Çarpma ve toplama (36), shift (37), toplama (45), OR (25).
Y5	Dizi elemanına store	37, 53	grades[i]'ye üretilen not (37) ve işaretli değer (53) yazılır.
Y6	const int SID ve hesapta kullanımı	4, 31	SID, LCG'nin başlangıç değeridir, bütün notlar ona bağlıdır.
Y7	Sonucun tutulduğu global değişken(ler)	7–10	bucket[5], mean, below_count, pass_count global değişkenlerdir.

1.5 Program çıktıları. Simülatörde çalıştırıldıktan sonra bellekten okunan değerler:

Global değişken	Adres (hex)	Değer	Ekran görüntüsü dosyası
SID	0x10010000	1240039 (0x0012EBE7)	screenshots/m1_data_hex.png, screenshots/m1_data_dec.png
grades[0..37]	0x10010004 – 0x10010098	38 not; ortalamanın altındakiler bit 8 ile işaretli (ör. 278 = 0x116 = işaretli 22)	screenshots/m1_data_hex.png, screenshots/m1_data_dec.png
bucket[0..4]	0x1001009C – 0x100100AC	17, 7, 6, 4, 4	screenshots/m1_data_hex.png, screenshots/m1_data_dec.png

mean	0x100100B0	51	screenshots/m1_data_hex.png, screenshots/m1_data_dec.png
below_count	0x100100B4	18	screenshots/m1_data_hex.png, screenshots/m1_data_dec.png
pass_count	0x100100B8	17	screenshots/m1_data_hex.png, screenshots/m1_data_dec.png

Not: Adresler RARS 1.6'nın veri yerleşimidir (.data 0x10010000'dan başlar).

1.6 İzlenecek kernel (12–20 dinamik instruction; en az bir load, store, ALU, koşullu branch ve call/return içermeli):

Alan	Değer
C satır aralığı	27 (mark dönüşü) ve 51–53 (3. döngü)
Başlangıç ve bitiş adresi (hex)	0x004000C0 – 0x004002E0
Dinamik instruction sayısı	20 instruction (ardışık değil: jr ra ile 0x004000CC → 0x00400288, alınan beq ile 0x004002A8 → 0x004002C8 atlanır)
Hangi döngü iterasyonu (ör. i = 3)	3. döngü, i = 1 (grades[1] = 58 ≥ mean = 51)

Öğretim üyesi (M1): ☐ Onaylandı ☐ Düzeltme istendi ☐ Reddedildi — Tarih: _____ Not: _____

Adım 2 — Assembly ve machine code (M2: 9 Kasım, Pazartesi)

2a. C → assembly eşlemesi (-O0; listenin tamamı depoda asm_O0.s):

C satır no	C ifadesi	Assembly satırları	Adres aralığı (hex)
13–14	int bucket_of(int v) { (giriş)	14–18	0x0040000C–0x0040001C
15–19	if (v < 50) … return 4;	19–43	0x00400020–0x00400070
20	} (dönüş)	45–49	0x00400074–0x00400084
23–24	int mark(int v, int m) { (giriş)	51–56	0x00400088–0x0040009C
25	if (v < m) return v | 0x100;	57–62	0x004000A0–0x004000B4
26	return v;	64	0x004000B8
27	} (dönüş)	66–70	0x004000BC–0x004000CC
29–30	int main(void) { (giriş)	72–75	0x004000D0–0x004000DC
31	unsigned int x = SID;	76–78	0x004000E0–0x004000E8
32	int sum = 0;	79	0x004000EC
35	for (int i = 0; i < N; i++)	80–81, 109–115	0x004000F0–0x004000F4, 0x00400164–0x00400178
36	x = 1664525u * x + 1013904223u;	83–90	0x004000F8–0x00400114
37	grades[i] = (x >> 16) % 101;	91–108	0x00400118–0x00400160
41	for (int i = 0; i < N; i++)	116–117, 150–156	0x0040017C–0x00400180, 0x00400210–0x00400224

42	int v = grades[i];	119–124	0x00400184–0x0040019C
43	bucket[bucket_of(v)]++;	125–136	0x004001A0–0x004001D4
44	if (v >= 59) pass_count++;	137–144	0x004001D8–0x004001FC
45	sum += v;	146–149	0x00400200–0x0040020C
47	mean = sum / N;	157–165	0x00400228–0x0040024C
50	for (int i = 0; i < N; i++)	166–167, 199–205	0x00400250–0x00400254, 0x004002E4–0x004002F8
51	int f = mark(grades[i], mean);	169–179	0x00400258–0x00400288
52	if (f != grades[i]) below_count++;	180–191	0x0040028C–0x004002C4
53	grades[i] = f;	193–198	0x004002C8–0x004002E0
56–57	return 0; }	206–211	0x004002FC–0x00400310

Not: Satır numaraları asm_O0.s'e, adresler RARS 1.6'daki yerleşime göredir (ilk 3 instruction RARS başlangıç bloğu; la/lla 2 instruction'a açılır).

Bölüm	Başlangıç adresi (hex)	Boyut (byte)	İçerik
.text	0x00400000	788 (197 instruction × 4)	Başlangıç bloğu, bucket_of, mark, main
.data / .bss	0x10010000	188	SID, grades[38], bucket[5], mean, below_count, pass_count

Von Neumann ilkesi bu adreslerde nasıl görünüyor? (1–2 cümle)

Instruction'lar (.text, 0x00400000'dan itibaren) ve veriler (.data, 0x10010000'dan itibaren) aynı 32-bit adres uzayında, aynı bellekte durur. İşlemci 0x004002A8 gibi bir adresten instruction'ı getirirken aynı bellek ve bus üzerinden lw/sw ile 0x10010008 gibi bir adresteki veriye erişir; ikisini ayıran tek şey adresleridir.

2b. -O0 ve -O1 karşılaştırması (kernel için):

Ölçü	-O0	-O1	Farkın nedeni
Kernel'deki instruction sayısı	20	4	mark inline edildi; call/ret ve prologue/epilogue kalktı
Load sayısı	7	1	-O0 i, f, ra, s0'ı stack'ten okur; -O1 yalnızca grades[i]'yi okur
Store sayısı	2	1	-O1'de f register'da kalır, yalnızca grades[i] yazılır
Kullanılan register sayısı	6 (ra, sp, s0, a0, a4, a5)	4 (s0, a1, a4, a5)	register allocation: değişkenler bellek yerine register'da tutulur

Not: -O1'de mark inline edildi; kernel'in karşılığı asm_O1.s 118–120 ve 114. Tüm iterasyon (i = 1): -O0'da 49 instruction (15 load, 7 store), -O1'de 6 instruction (1 load, 1 store).

2c. Elle machine code (en az 10 instruction; x86-64 için 8). Kullanılan tüm formatlar kapsanmalı.

#	Adres (hex)	Assembly	Format	Alanlar (ad = binary)	Machine code (hex)	Bellekteki byte sırası	objdump ile aynı mı?
1	0x00400 0E0	lui x15,0x12f	U	imm[31:12]=00000 000000100101111, rd=01111, opcode=0110111	0x0012F7B 7	B7 F7 12 00	Evet (RARS)
2	0x00400 0E4	addi x15,x15,-1049	I	imm[11:0]=101111 100111, rs1=01111, funct3=000, rd=01111, opcode=0010011	0xBE77879 3	93 87 77 BE	Evet (RARS)
3	0x00400 0E8	sw x15,-20(x8)	S	imm[11:5]=111111 1, rs2=01111, rs1=01000, funct3=010, imm[4:0]=01100, opcode=0100011	0xFEF4262 3	23 26 F4 FE	Evet (RARS)
4	0x00400 284	jal x1,-508	J	imm[20]=1, imm[10:1]=110000 0010, imm[11]=1, imm[19:12]=11111 111, rd=00001, opcode=1101111	0xE05FF0E F	EF F0 5F E0	Evet (RARS)

#	Adres (hex)	Assembly	Format	Alanlar (ad = binary)	Machine code (hex)	Bellekteki byte sırası	objdump ile aynı mı?
5	0x00400298	slli x15,x15,2	I	funct7=0000000, shamt=00010, rs1=01111, funct3=001, rd=01111, opcode=0010011	0x00279793	93 97 27 00	Evet (RARS)
6	0x0040029C	add x15,x14,x15	R	funct7=0000000, rs2=01111, rs1=01110, funct3=000, rd=01111, opcode=0110011	0x00F707B3	B3 07 F7 00	Evet (RARS)
7	0x004002A4	lw x14,-40(x8)	I	imm[11:0]=111111011000, rs1=01000, funct3=010, rd=01110, opcode=0000011	0xFD842703	03 27 84 FD	Evet (RARS)
8	0x004002A8	beq x14,x15,32	B	imm[12]=0, imm[10:5]=000001, rs2=01111, rs1=01110, funct3=000, imm[4:1]=0000, imm[11]=0, opcode=1100011	0x02F70063	63 00 F7 02	Evet (RARS)
9	0x004002E0	sw x14,0(x15)	S	imm[11:5]=0000000, rs2=01110, rs1=01111, funct3=010, imm[4:0]=00000, opcode=0100011	0x00E7A023	23 A0 E7 00	Evet (RARS)
10	0x004000CC	jalr x0,0(x1)	I	imm[11:0]=000000000000, rs1=00001, funct3=000, rd=00000, opcode=1100111	0x00008067	67 80 00 00	Evet (RARS)

Doğrulama: screenshots/m2_enc_1.png (#1–3, #10) ve screenshots/m2_enc_2.png (#4–9), RARS 1.6 Text Segment, Code sütunu.

objdump: objdump_O0.txt (xPack gcc 15.2.0, -O0, objdump -d -r). Nesne dosyası bağlanmadığı için adresler 0'dan başlar; #1–3 ve #5–10 aynı machine code ile görünür. #4 jal, .o dosyasında auipc + jalr olarak durur ve bağlama sırasında jal'e dönüşür.

Zorunlu instruction'ların tablodaki sıra numarası:

- X SID'i yükleyen instruction(lar): #1, #2
- X Koşullu branch: #8
- X Load: #7
- X Store: #3, #9
- X Fonksiyon çağrısı: #4

SID'in yüklenmesi. Sabitin neden tek instruction'a sığmadığı, nasıl bölündüğü ve gerekiyorsa +1 düzeltmenin sign extension ile açıklaması:

SID = 1240039 = 0x0012EBE7, 12 bitlik addi immediate'ine sığmadığı için lui (üst 20 bit) + addi (alt 12 bit) çiftine bölünür. Alt 12 bit 0xBE7'dir; bit 11'i 1 olduğundan addi bunu sign extension ile negatif okur: 0xBE7 − 4096 = −1049. Bu yüzden lui'ye 0x12E yerine 0x12F (+1 düzeltmesi) yüklenir: 0x12F × 4096 − 1049 = 1241088 − 1049 = 1240039.

Branch offset hesabı. Branch'in adresi, hedef adres ve offset'in adım adım hesabı:

beq x14,x15 adresi 0x004002A8, hedefi .L17 = 0x004002C8. Offset = hedef − PC = 0x004002C8 − 0x004002A8 = 0x20 = 32 byte (8 instruction ileri). B formatında bit 0 saklanmaz; 32 = 0b0_0000_0010_0000 → imm[12]=0, imm[11]=0, imm[10:5]=000001, imm[4:1]=0000.

Adım 3 — CPU, interrupt ve bus (final paketiyle: 16 Kasım, Pazartesi)

Model varsayımları (okudum ve uyguladım):

- ☐ Pipeline yok, her an tek instruction yürütülür.
- ☐ PC, IR, MAR ve MBR Fig. 3.2'deki gibi kavramsal olarak vardır.
- ☐ Her bellek erişimi bir bus işlemidir; cache bu adımda yok sayılır.
- ☐ Data bus genişliği = ____ bit.

Modelin gerçek donanımdan ayrıldığı yerler (3–5 cümle):

3a. Instruction cycle izleme tablosu. Her dinamik instruction bir satırdır. Bir instruction birden çok bellek erişimi yapıyorsa MAR, MBR ve bus hücrelerine sırayla yazılır. Kategori: PM = processor–memory, PIO = processor–I/O, DP = data processing, C = control.

#	PC (önce → sonra)	Assembly	IR (hex)	Fig. 3.6 durumları	MAR	MBR	ALU işlemi	Değişen register	Bus (adres / data / control)	Kategori
1										
2										
3										
4										
5										
6										
7										
8										
9										
10										
11										
12										
13										
14										

15										
16										
17										
18										
19										
20										

Kernel toplamları	Sayı
Instruction fetch	
Operand fetch (of)	
Operand store (os)	
PC'nin sıra dışı değiştiği instruction	

3b. Fig. 3.4 makinesine aktarım. Kernel'in tek bir dizi elemanını güncelleyen parçası, opcode 1, 2, 3, 5 ve 7 ile yeniden yazılır ve Fig. 3.5 formatında izlenir:

Adım	Fetch / execute	PC	IR (hex)	AC	Değişen bellek hücresi
1					

2					
3					
4					
5					
6					
7					
8					

Ölçü	Seçilen ISA	Fig. 3.4 makinesi
Aynı iş için instruction cycle sayısı		

Tek accumulator'ın getirdiği sınırlama (PDP-11 `ADD B,A` ile karşılaştırarak):

IAS register'ları ve seçilen ISA:

IAS register'ı	IAS'taki görevi	Seçilen ISA'daki karşılığı	Programcıya görünür mü?
MBR			
MAR			
IR			
IBR			
PC			
AC			
MQ			

3c. Interrupt senaryosu (k = ____, tür = ____):

Soru	Cevap
Interrupt hangi instruction'ın execute'u sırasında geliyor? (adres, assembly)	
Ne zaman tanınır ve bu instruction neden yarıda kesilmez? (Fig. 3.9)	
Kaydedilen PC değeri (hex)	
Kaydedilen diğer durum bilgisi	

Seçilen ISA'daki gerçek mekanizma (register adları, vektör)	
Kaynak (ISA referansı, bölüm, sayfa)	
Handler dönüşünde ilk çalışan instruction'ın adresi	

Fonksiyon çağrısı (Y3) ve interrupt karşılaştırması:

Ölçüt	Fonksiyon çağrısı	Interrupt
Akışı kim başlatır?		
Dönüş adresini kim kaydeder, nereye?		
User program bundan haberdar mı?		

İkinci (daha yüksek öncelikli) interrupt. Aynı olay dizisini iki yaklaşımla zaman çizelgesi olarak doldurun (Fig. 3.13 ve 3.14):

Zaman	Sequential (disable)	Nested (priority)
t_0: 1. interrupt gelir		
t_1: 2. interrupt gelir		
t_2		
t_3		
t_4: user program'a dönüş		

3d. Bus ve interconnection.

Bus işlemi türü (kernel, cache yok)	Sayı	Aktif control line'lar
Instruction fetch (read)		
Data read		
Data write		
Toplam		

Hesap	Değer	Adımlar
Address bus genişliği ve maksimum adreslenebilir bellek		
Data bus 16 bit olsaydı kernel'in bus cycle sayısı		
Mevcut / 16-bit oranı		

PCIe paket verimi (N × 4 = ____ byte'lık tek Memory Write TLP; Fig. 3.25):

Alan (octet)	12-byte header, ECRC yok	16-byte header, ECRC var
STP framing (baş)		
Sequence number		
Header		
Data		
ECRC		
LCRC		
STP framing (son)		
Toplam		
Verim = data / toplam		
128b/130b ile verim		

Sonucun bir çekirdekten PCIe cihazına yolu (Fig. 3.17 ve 3.21): her adımda bağlantı, protokol katmanı ve aktarım birimi.

Adım 4 — Cache simülasyonu (final paketiyle: 16 Kasım, Pazartesi)

4a. Data adres izi. Tüm load ve store'lar depoda `trace.csv` dosyasındadır. Biçim: `sira,RW,adres_hex,boyut,c_satiri,stack_mi`.

Ölçü	Değer
Kullanılan derleme `-O0` / `-O1`) (	
Toplam data referansı	

Load / store	
Stack erişimi sayısı	
Ana diziye erişim sayısı	

4b. İlk 5 farklı adresin alanları:

Adres (hex)	Adres (binary)	Tag	Line / Set	Word

4c. Elle simülasyon (ilk 30 referans):

#	R/W	Adres (hex)	Tag	Line / Set	Word	Hit / Miss	Yerinden çıkan blok	Bu blok daha önce cache'te miydi?
1								
2								
3								
4								
5								
6								
7								
8								
9								
10								
11								
12								
13								
14								
15								
16								
17								
18								
19								
20								
21								
22								
23								
24								

#	R/W	Adres (hex)	Tag	Line / Set	Word	Hit / Miss	Yerinden çıkan blok	Bu blok daha önce cache'te miydi?
25								
26								
27								
28								
29								
30								

Cache'in 30. referanstan sonraki anı. Line sayınız 8'den fazlaysa satır ekleyin.

Line	Set	Valid	Tag (hex)	Dirty (write-back)	LRU sırası (associative)
0					
1					
2					
3					
4					
5					
6					
7					

4d. Tam simülasyon sonuçları:

Ölçü	Elle (ilk 30)	Araç (ilk 30)	Araç (tüm iz)
Referans	30	30	
Hit			
Miss			
Hit ratio H			
Ana belleğe yazma sayısı			

Kullanılan araç ve ayarları (kendi simülatörünüz varsa dosya adı):

- ☐ Elle ve araçla yapılan simülasyon ilk 30 referansta birebir örtüşüyor.

4e. Tasarım uzayı (cache boyu sabit):

Mapping (line boyu sabit = ____ B)	Hit	Miss	H
Direct			
2-way set-associative			
4-way set-associative			
Fully associative (LRU)			

Line boyu (mapping sabit = ____)	Hit	Miss	H
8 B			
16 B			
32 B			
64 B			

Grafik dosyası ve eğilimlerin locality ile açıklaması (thrashing varsa çatışan adresler):

4f. Ortalama erişim süresi: $T_s = H \times T1 + (1 - H) \times (T1 + T2)$

Durum	H	T_s (cycle)	Speedup (cache'siz duruma göre)
Cache yok	0	T2 =	1
Kişiye özel cache			

4g. Kod değişikliği:

	Önce	Sonra
Değiştirilen C satır(lar)ı		
Y7 çıktısı (aynı olmalı)		
H		

T_s (cycle)		

Değişikliğin gerekçesi; H artmadıysa nedeni:

Adım 5 — Performans ve architecture/organization (final paketiyle: 16 Kasım, Pazartesi) 5a-1. Dinamik instruction dağılımı ve CPI (CPI_i değerleri yönergedeki model varsayımıdır):

Sınıf	CPI_i (varsayım)	I_ i - (O0)	I_-) i O (1
ALU	1		
Çarpma	3		
Load	1		
Store	1		
Branch / jump	2		
Ic (toplam)			
Ortalama CPI			

5a-2. Süre ve MIPS rate (f = ____ GHz, τ = ____ ns):

Ölçü	Formül	-O0	-O1
T (bellek etkisi yok)	Ic × CPI × τ		
MIPS rate	f / (CPI × 10^6)		
m (instruction başına data referansı)	data ref. / Ic		
T, cache yok (k = T2)	Ic × [p + (m × k)] × τ		
T, cache var (k = T_s)	Ic × [p + (m × k)] × τ		

Karşılaştırma	Değer
Speedup -O0 → -O1 (Eq. 2.1, cache var)	

Speedup cache yok → cache var -O0) (	

MIPS rate'in iki derlemeyi karşılaştırmada neden yanıltıcı olabileceği, kendi sayılarınızla:

5a-3. Amdahl yasası (Nc = ____):

Alan	Değer
Paralelleştirilebilen kısım (C satırları)	
Bağımsızlık gerekçesi (veri bağımlılığı)	
f (yürütme süresindeki pay)	
Speedup = 1 / [(1 − f) + f / Nc]	
Nc → ∞ limiti	

5b. Architecture ve organization (en az 10 özellik):

#	Özellik	Architecture / Organization	Gerekçe (Stallings tanımı)	Değişseydi Y7 değişir miydi? Kanıt
1				
2				
3				
4				
5				
6				
7				
8				
9				
10				

Ch. 1.2'deki dört temel fonksiyon:

Fonksiyon	Kernel'den örnek instruction (adres, assembly)	Açıklama
Data processing		

Data storage		
Data movement		
Control		

Final teslim (16 Kasım Pazartesi; sözlü savunma 20 Kasım Cuma)

Teslim kontrol listesi:

- ☐ Bu tutanak, tüm adımları dolu olarak (PDF ve düzenlenebilir sürüm)
- ☐ C kaynak kodu (`main.c`) ve sonraki 4g sürümü (`main_v2.c`)
- ☐ `asm_O0.s`, `asm_O1.s` ve `objdump -d` çıktısı
- ☐ `trace.csv` ve cache simülasyon çıktıları (kendi simülatörünüz varsa kaynak kodu)
- ☐ Ekran görüntüleri: machine code doğrulaması, program çıktıları, cache durumu
- ☐ 4e grafiği
- ☐ Git deposunda M1, M2 ve final etiketleri

Kilometre taşı kaydı (öğretim üyesi doldurur):

Kilometre taşı	Son tarih	Teslim tarihi	Git etiketi	Paraf
M1 Öneri (Adım 0–1)	2 Kasım (Pzt)			
M2 Assembly ve machine code (Adım 2)	9 Kasım (Pzt)			
M3 CPU, interrupt, bus (Adım 3)	16 Kasım (Pzt, final paketiyle)			
M4 Cache ve performans (Adım 4–5)	16 Kasım (Pzt, final paketiyle)			
Final ve sözlü savunma	Teslim 16 Kasım (Pzt); savunma 20 Kasım (Cuma)			

YZ kullanım beyanı. Kullanılmadıysa ilk satıra "Kullanılmadı" yazın. Beyan edilmemiş kullanım akademik dürüstlük ihlalidir.

Araç ve sürümü	Adım	Ne için kullanıldı	Örnek prompt	Çıktıyı nasıl doğruladım
Claude Opus 5.5 (Anthropic)	Adım 0	Parametre formüllerinin açıklanması ve hesabın kontrolü	“Numaram 5001240039, parametreleri nasıl hesaplarım?”	Tüm parametreleri yönergedeki formüllerle kendim yeniden hesapladım, yönergedeki örnek numarayla kontrol ettim

Claude Opus 5.5 (Anthropic)	Adım 1	C kodunu yazarken takıldığım yerlerde soru sordum (LCG, döngü yapısı); görev seçimi ve tasarım kararları benim	"LCG'yi unsigned int ile nasıl yazarım?"	godbolt'ta -O0 ile derleyip çarpmanın mul olarak çıktığını ve kütüphane çağrısı olmadığını kontrol ettim
Claude Opus 5.5 (Anthropic)	Adım 1	gcc çıktısının RARS 1.6'da çalışması için hangi değişikliklerin gerektiğini sordum (direktifler, lla→la, call→jal); uyarlamayı kendim yaptım	"gcc'nin ürettiği assembly'yi RARS'ta çalışır hale nasıl getiririm?"	RARS'ta çalıştırıp register ve bellek değerlerini C kodunun beklenen sonuçlarıyla karşılaştırdım
Claude Opus 5.5 (Anthropic)	Adım 2	2a eşleme tablosunun ve 2b karşılaştırmasının nasıl doldurulacağını sordum; tabloları kendim doldurdum	"2a nasıl dolduruyoruz?"	Satır numaralarını asm_O0.s / asm_O1.s'te, adresleri RARS Text Segment'te kontrol ettim
Claude Opus 5.5 (Anthropic)	Adım 2	Encoding'leri kendim yaptım; takıldığım yerlerde (B/J formatı, SID'deki +1 düzeltmesi, branch offset) soru sordum ve sonuçlarımı kontrol ettirdim	"beq'te offset nasıl hesaplanıyor?", "addi'de neden üst 20 bite +1 ekleniyor?"	Alan tablolarını ve hex değerlerini elle çıkardıktan sonra 10 machine code'u RARS'ın Code sütunuyla tek tek karşılaştırdım (m2_enc_1.png, m2_enc_2.png)

Öğrenci beyanı. Bu tutanaktaki C kodunu, elle machine code kodlamasını, instruction cycle izleme tablosunu ve elle cache simülasyonunu kendim yaptım. Başka bir öğrencinin çalışmasını kopyalamadım ve kendi çalışmamı paylaşmadım. YZ kullanımımı eksiksiz beyan ettim. Sözlü savunmada bu çalışmayı açıklayamamamın puanımı etkileyeceğini biliyorum.

Ad Soyad	Öğrenci numarası	Tarih	İmza
Necati Selçuk	5001240039	8.11.2026	

Değerlendirme (öğretim üyesi doldurur)

Bileşen	Azami	Puan	Not
A1 Öneri ve parametre hesabı (Adım 0–1)	5		
A2 Assembly ve machine code (Adım 2)	15		

A3 CPU, interrupt, bus (Adım 3)	25		
A4 Cache (Adım 4)	25		
A5 Performans ve architecture/organization (Adım 5)	10		
Tutanak ve rapor kalitesi	5		
Sözlü savunma	15		
Geç teslim kesintisi	–		
Toplam	**100**		

Sözlü savunma kaydı:

Madde	Sorulan	Sonuç (başarılı / kısmen / başarısız)	Puan
1. Canlı encode -O1 listesinden) (			
2. Adres alanları ve hit/miss			
3. Interrupt k+1'de gelseydi			
4. Tasarım kararı savunması			

☐ 1. ve 2. maddelerin ikisi de başarısız: A2–A4 puanları 0.5 ile çarpıldı.

Öğretim üyesi	Tarih	İmza